SUNWAY COLLEGE

## FOUNDATION IN ARTS

## ASSIGNMENT 1 COVER SHEET

| | |
|---|---|
| **SUBJECT TITLE** | **: INTRODUCTION TO PROGRAMMING** |
| **SUBJECT CODE** | **: PCSC002** |
| **COURSEWORK TITLE** | **: INDIVIDUAL WORK** |
| **COURSEWORK** | **: 15%** |
| **DUE DATE** | **: 6$^{TH}$ Sept 2023** |

The objectives of this assignment are:

- To encourage students to think critically about problem solving in programming.
- To apply the knowledge of Java programming.
- To practice good programming techniques.

The learning outcomes of this assignment is:

- To be able to create basic Java applications.
- To be able to use decision making constructs for problem solving.

Student Name : Chiam Jie Zu
Student ID : 23026628

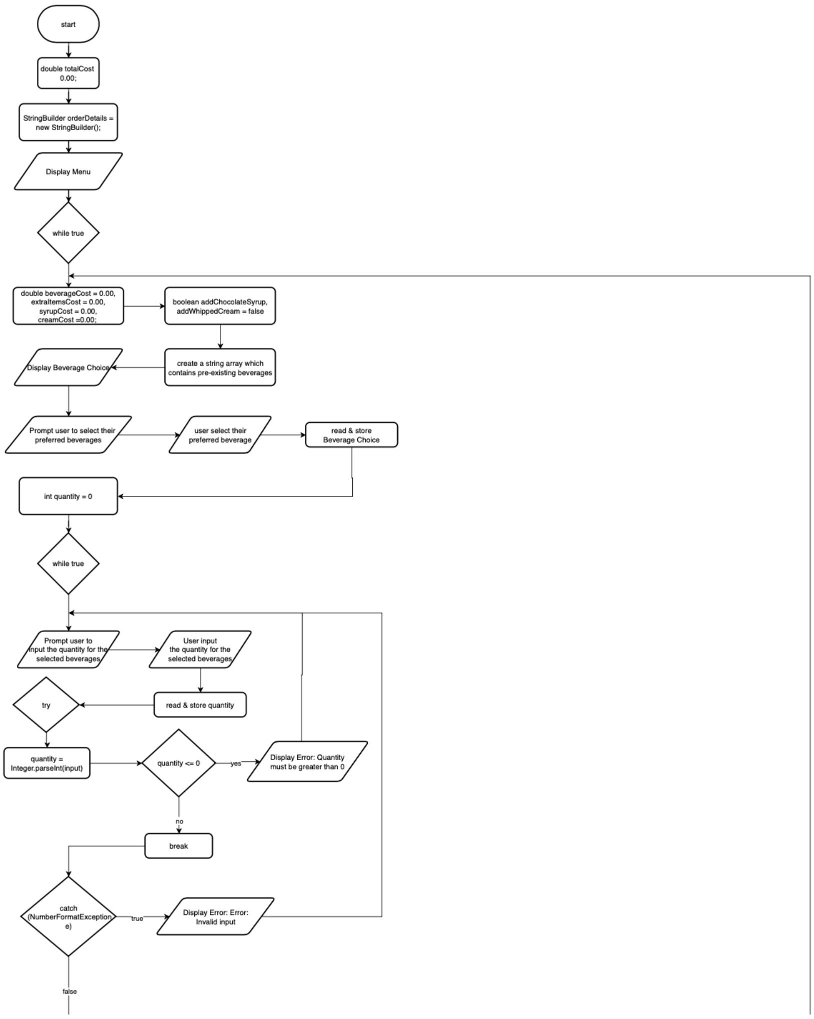
**INSTRUCTIONS:**

1. Documentation must be submitted as softcopy through eLearn and must include the following:
   a. Flowchart for each question using Microsoft Visio/ draw.io/ Lucidchart.
   b. Source codes for each question.
   c. Screenshot of input and output for each relevant data.
2. Students are required to display good programming practice through their coding style.
3. Answer all questions.

**TASKS:**

1. A Coffee shop is offering different types of beverages. The menu includes regular coffee for $5.50, cappuccino for $7.49, and a latter for $8.00. Customers can request extra items such as whipped cream for $1.25 and chocolate syrup for $2.00. Implement a program which allows customers to select their preferred beverages and extra items, then display the total cost. Higher marks will be given for programs created using JOptionPane.

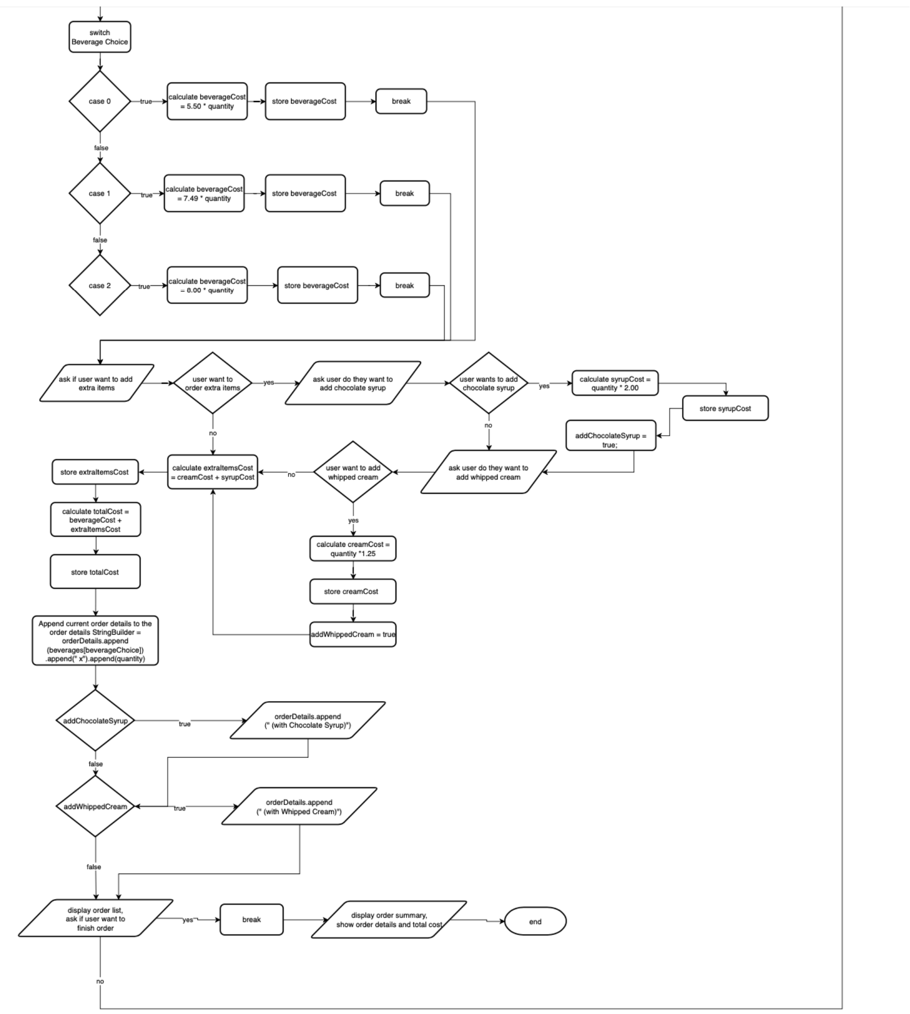
Flowchart : https://t.ly/ezEHS (link to view the flowchart)




**Commented [KTST1]:** 1. Some problem at the try exception. Based on the flow, your try is a diamond with only 1 choice. Try except works by this rule:

If there is no error, then the code procees and skips catch.

If there is an error, it will jump to the exception.


switch
Beverage Choice
case 0
true
calculate beverageCost = 5.50 * quantity
store beverageCost
break
false
case 1
true
calculate beverageCost = 7.49 * quantity
store beverageCost
break
false
case 2
true
calculate beverageCost = 8.00 * quantity
store beverageCost
break
ask if user want to add extra items
user want to order extra items
yes
ask user do they want to add chocolate syrup
user wants to add chocolate syrup
yes
calculate syrupCost = quantity * 2.00
store syrupCost
addChocolateSyrup = true;
no
ask user do they want to add whipped cream
user want to add whipped cream
no
calculate extraItemsCost = creamCost + syrupCost
yes
calculate creamCost = quantity *1.25
store creamCost
addWhippedCream = true
store extraItemsCost
calculate totalCost = beverageCost + extraItemsCost
store totalCost
Append current order details to the order details StringBuilder = orderDetails.append (beverages[beverageChoice]) .append(" x").append(quantity)
addChocolateSyrup
true
orderDetails.append (" (with Chocolate Syrup)")
false
addWhippedCream
true
orderDetails.append (" (with Whipped Cream)")
false
display order list, ask if user want to finish order
yes
break
display order summary, show order details and total cost
end
no

Task 1 source codes:

Commented [KTST2]: Code is very good.

```
/*The purpose of this program is to create a menu that display types of beverage for a coffee shop
 *  which allows customers to select their preferred beverages and extra items, then display the total cost.*/

import javax.swing.JOptionPane;

public class CoffeeShopMenu {
    public static void main(String[] args) {
        double totalCost = 0.00;
        StringBuilder orderDetails = new StringBuilder(); // Initialize StringBuilder for order details

        //Display the menu
        JOptionPane.showMessageDialog(null, "\nBeverages:" + "\nRegular Coffee $5.50 \nCappuccino $7.90 \nLatte $8.00"
                                        + "\nExtra Items: \nWhipped Cream $1.25 \nChocolate Syrup $2.00","Coffee Shop Menu",1);

        while (true) {
            double beverageCost = 0.00, extraItemsCost = 0.00, syrupCost = 0.00, creamCost = 0.00;
            boolean addChocolateSyrup = false, addWhippedCream = false;

            String[] beverages = {"Regular Coffee $5.50", "Cappuccino $7.49", "Latte $8.00"};
            int beverageChoice = JOptionPane.showOptionDialog(null, "Select your preferred beverages:", "Coffee Menu", JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE,
                                null, beverages,beverages[0] );

            //ask user to enter quantity for selected beverage
            int quantity = 0;
            while (true) {
                String input = JOptionPane.showInputDialog(null,"How many cups of " + beverages[beverageChoice] + " do you want to order?","Order Quantity",1);
                try {
                    quantity = Integer.parseInt(input);
                    if (quantity <= 0) {
                        JOptionPane.showMessageDialog(null, "Error: Quantity must be greater than 0.","ERROR",0);
                    } else {
                        break;
                    }
                } catch (NumberFormatException e) {
                    JOptionPane.showMessageDialog(null, "Error: Invalid quantity. Please enter a valid quantity.","Error",0);
                }
            }
```

```
switch (beverageChoice) {
    case 0:
        beverageCost = 5.50 * quantity;
        break;
    case 1:
        beverageCost = 7.49 * quantity;
        break;
    case 2:
        beverageCost = 8.00 * quantity;
        break;
}

//ask user do they want to add extra items
int extraItemsChoice = JOptionPane.showConfirmDialog(null,"Do you want to add
extra items? \nWhipped Cream $1.25 \nChocolate Syrup $2.00",
    "Extra Items Menu",JOptionPane.YES_NO_OPTION);

if (extraItemsChoice == JOptionPane.YES_OPTION) {
    int addChocolateSyrupChoice = JOptionPane.showConfirmDialog(null, "Do you want
to add chocolate syrup $2.00?", "Chocolate Syrup", JOptionPane.YES_NO_OPTION);
    if (addChocolateSyrupChoice == JOptionPane.YES_OPTION) {
        syrupCost = quantity * 2.00;
        addChocolateSyrup = true;
    }
    int addWhippedCreamChoice = JOptionPane.showConfirmDialog( null, "Do you
want to add whipped cream $1.25?","Whipped Cream",JOptionPane.YES_NO_OPTION);
    if (addWhippedCreamChoice == JOptionPane.YES_OPTION) {
        creamCost = quantity* 1.25;
        addWhippedCream = true;
    }

}

//calculate extra items cost and total cost
extraItemsCost = creamCost + syrupCost;
totalCost += beverageCost + extraItemsCost;

            // append order details to the orderDetails StringBuilder
orderDetails.append(beverages[beverageChoice]).append(" x").append(quantity);
if (addChocolateSyrup) {
    orderDetails.append(" (with Chocolate Syrup)");
}
if (addWhippedCream) {
    orderDetails.append(" (with Whipped Cream)");
}
orderDetails.append("\n");

//Display order list, and ask user do they want to finish their order.
```

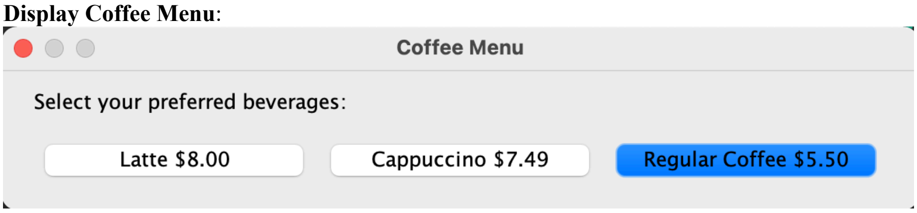
```
        int finishOrder = JOptionPane.showConfirmDialog( null, "Order list: " +
orderDetails.toString() + "\nDo you want to finish the order? "
                              + "\n (click no to continue the order)", "Order List",
JOptionPane.YES_NO_OPTION );
        if (finishOrder == JOptionPane.YES_OPTION) {
          break;
        }
      }
    // Display the order summary including order details and total cost
      JOptionPane.showMessageDialog(null, "Order Summary:\n" + orderDetails.toString() +
"Total cost: $" + String.format("%.2f", totalCost),
      "Order Summary", JOptionPane.INFORMATION_MESSAGE);
    }
}
```

Task 1 Input and Output:

**Display Coffee Shop Menu:**

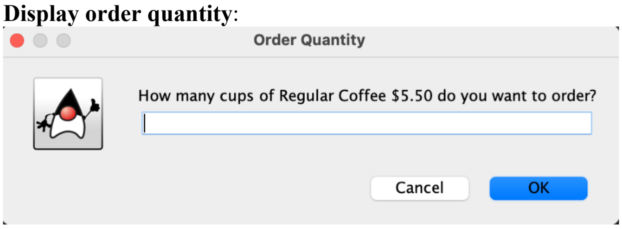

(if user press ok, the program will display coffee menu, prompt user to select their preferred beverage)

**Display Coffee Menu**:

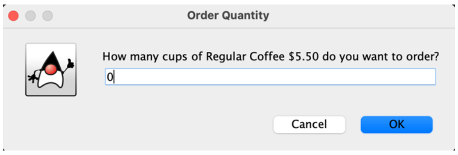


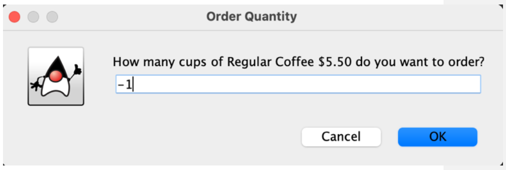

(after user select their preferred beverage, the program will display the order quantity)

**Display order quantity**:

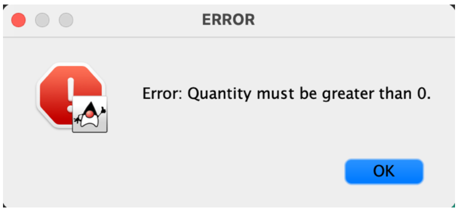


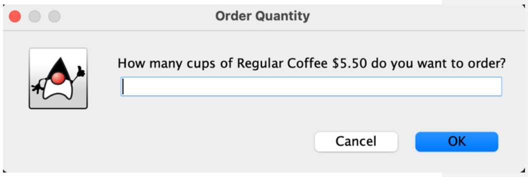
(if user enter 0 or any negative value for the for the order quantity)



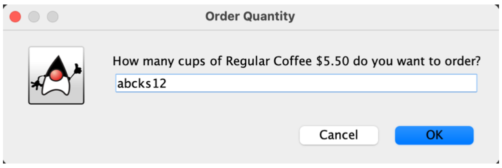



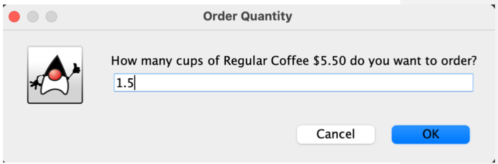

**Display error**: (The program will display error, then prompt user to enter the order quantity again.)

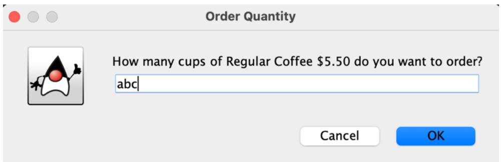


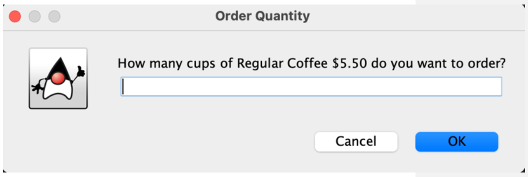

(if user enter string / decimal value for the order quantity)



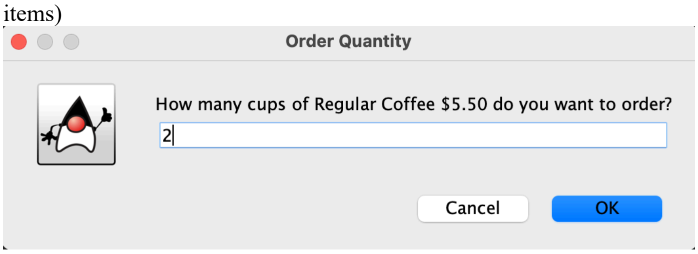



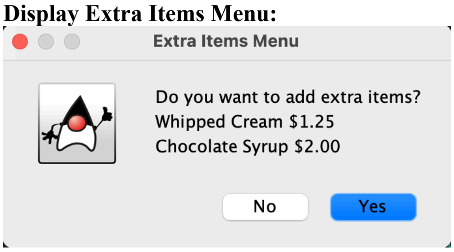
**Display error**: (The program will display error, then prompt user to enter the order quantity again)


(if user enter the correct amount of quantity, then the program will ask if user want to add extra items)

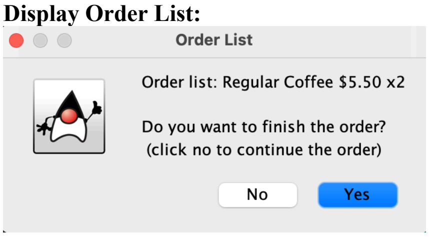



**Display Extra Items Menu:**



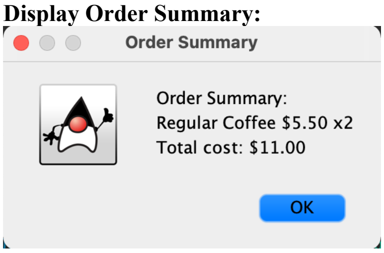

(if user click no, they don't want to add extra items)

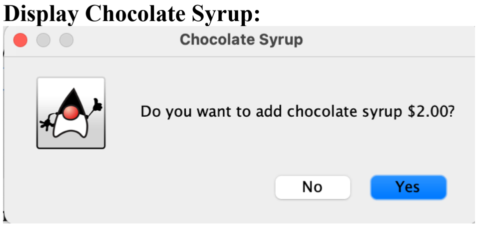
**Display Order List:**




(if user click yes to finish order)

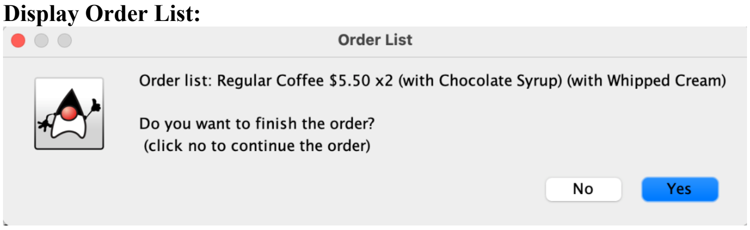
**Display Order Summary:**

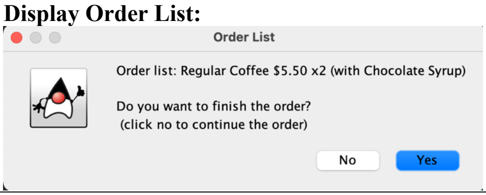


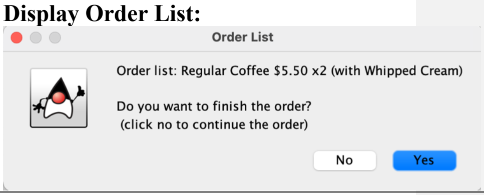
(if user click yes, they want to add extra items)
**Display Chocolate Syrup:**



(if user click yes, they want to add chocolate syrup) / (if user click no, they don't want to add chocolate syrup)
**Display Whipped Cream:**

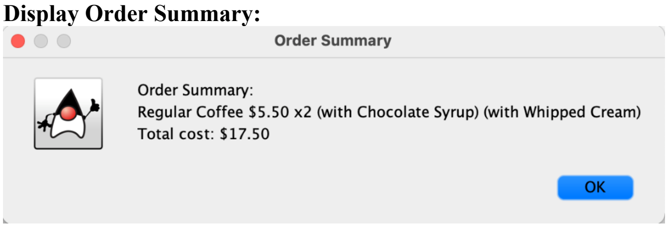


(if user click yes, they want to add whipped cream) / (if user click no, they don't want to whipped cream)
**Display Order List:**


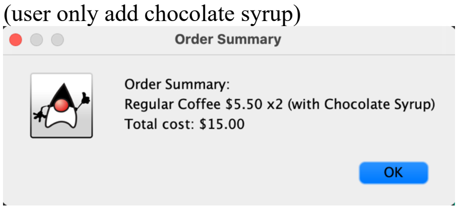

(user only add chocolate syrup)
**Display Order List:**



(user only add whipped cream)
**Display Order List:**


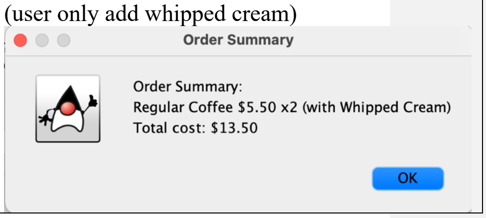
(if user click yes to finish the order)
**Display Order Summary:**



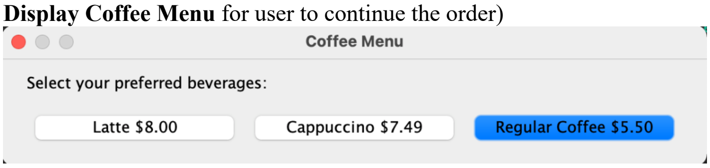

**Display Order Summary:**
(user only add chocolate syrup)



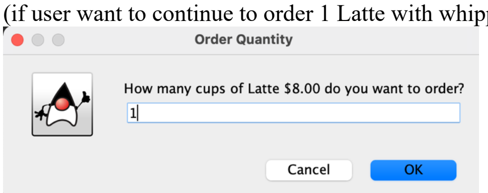

**Display Order Summary:**
(user only add whipped cream)



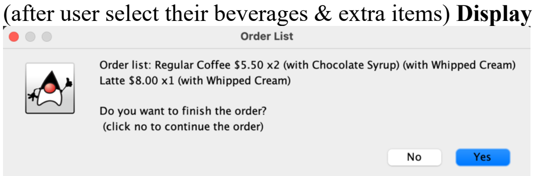

(if user click no to finish the order, they want to order another beverage, the program will **Display Coffee Menu** for user to continue the order)




(if user want to continue to order 1 Latte with whipped cream)

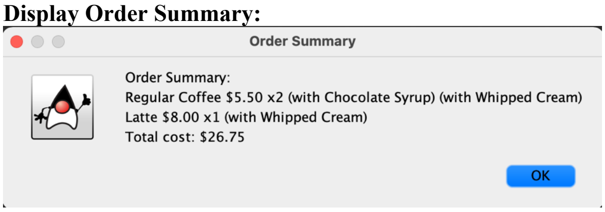



(after user select their beverages & extra items) **Display Order List:**



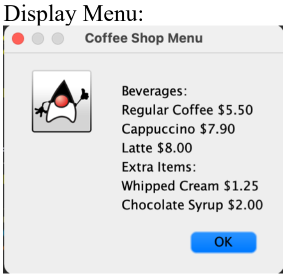
(if user click yes to finish the order)

**Display Order Summary:**

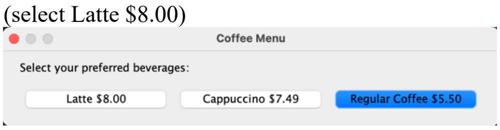



**Testing:**

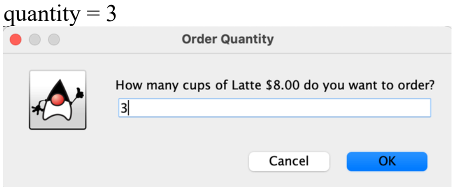
Display Menu:



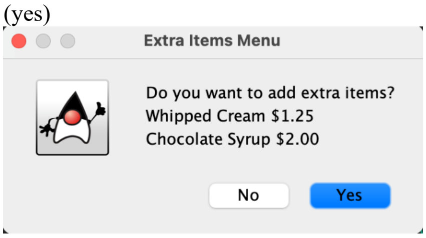

(select Latte $8.00)



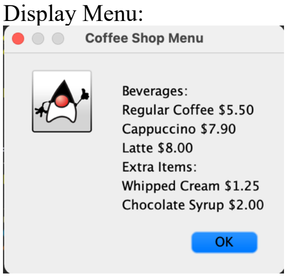

quantity = 3



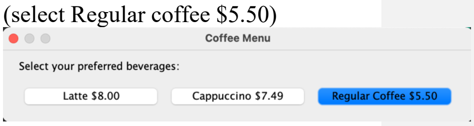

(yes)



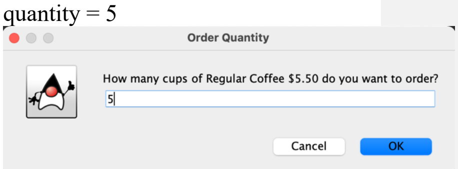

Display Menu:



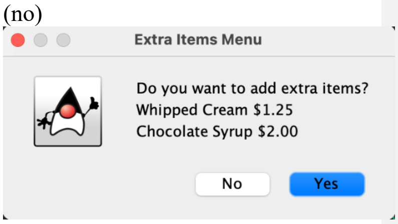

(select Regular coffee $5.50)



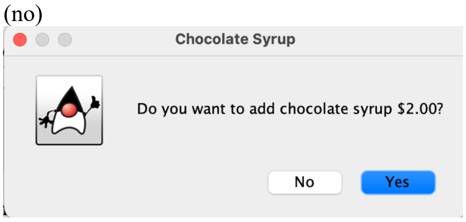

quantity = 5



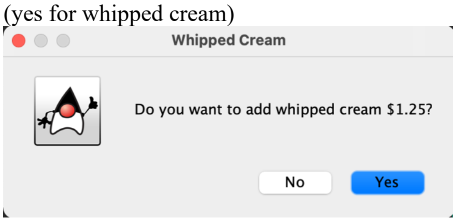

(no)



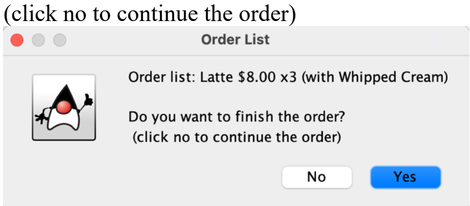
(no)




(yes for whipped cream)

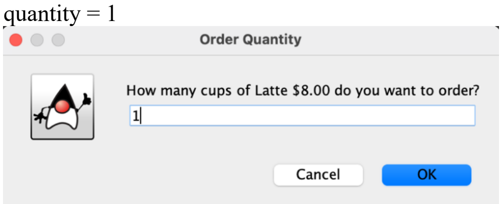



(click no to continue the order)

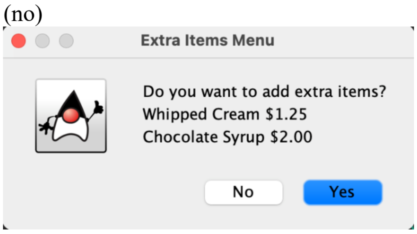



(continue the order, select Latte $8.00)

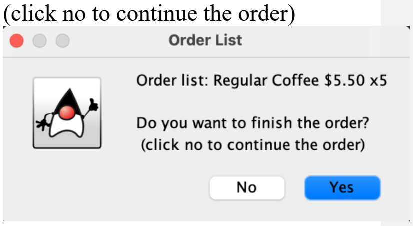

quantity = 1



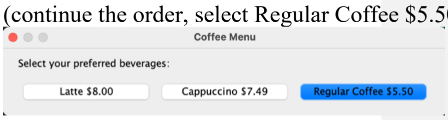

(no)



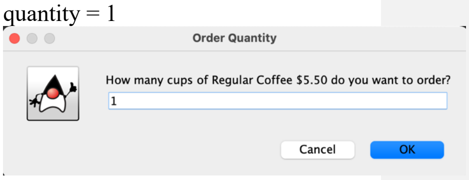

(click no to continue the order)



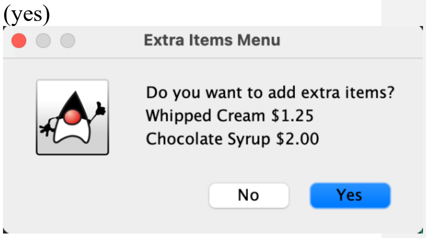

(continue the order, select Regular Coffee $5.50)



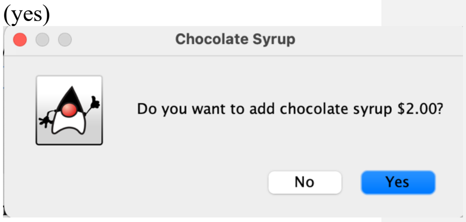

quantity = 1



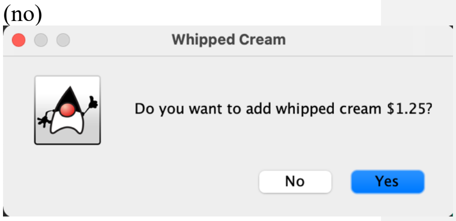

(yes)



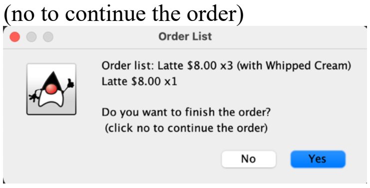

(yes)




(no)

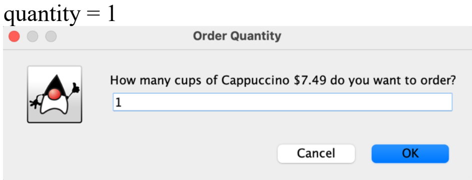


(no to continue the order)

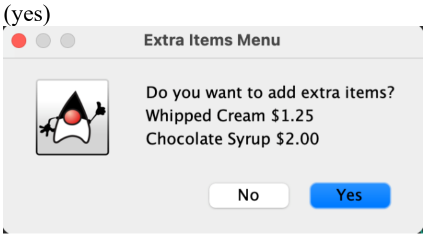



(continue order, select Cappuccino $7.49)

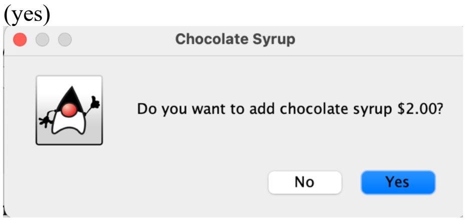

quantity = 1



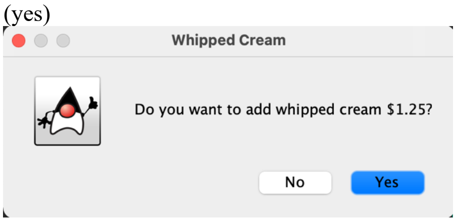

(yes)



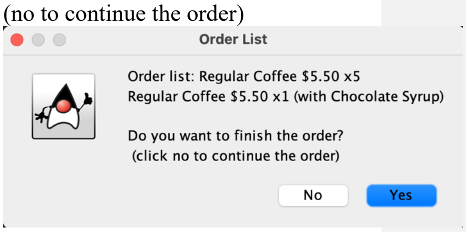

(yes)




(yes)

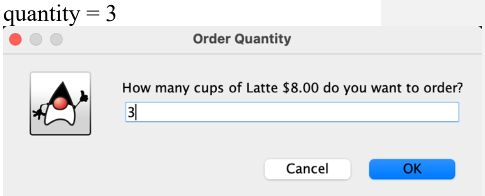



(no to continue the order)

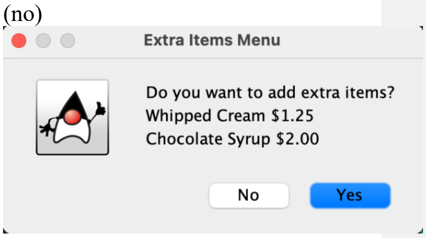



(select Latte $8.00)

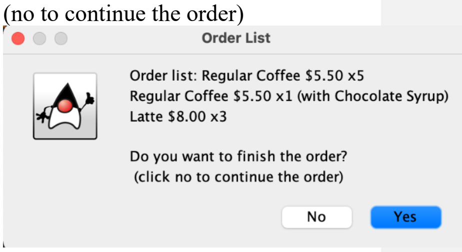

quantity = 3



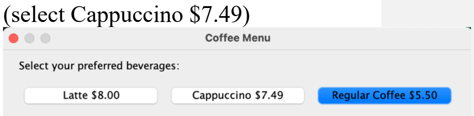

(no)




(no to continue the order)

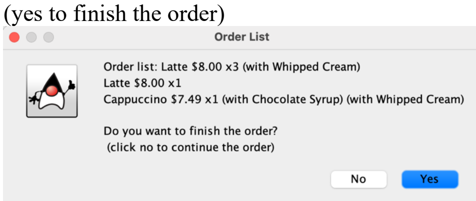



(select Cappuccino $7.49)

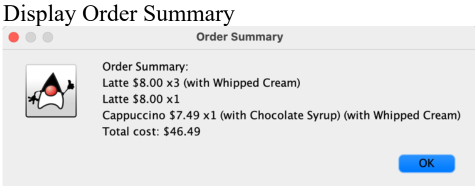


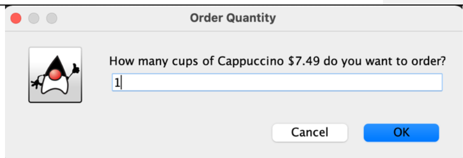

quantity = 1

(yes to finish the order)

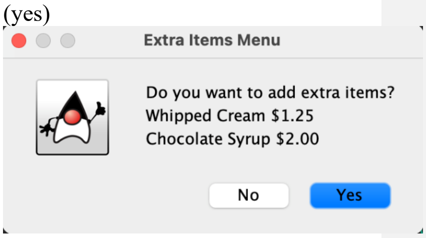



**Output 1:**
Display Order Summary

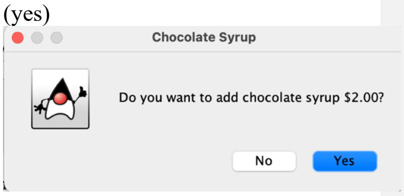


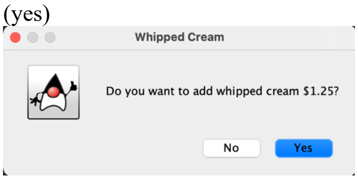

(yes)



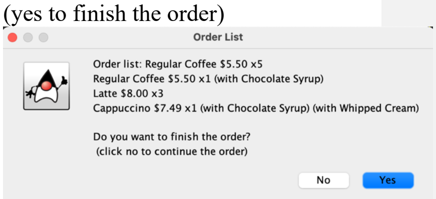

(yes)



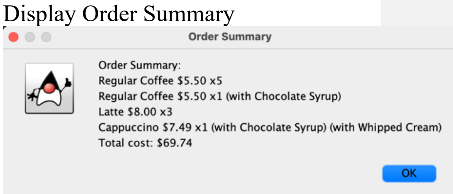

(yes)




(yes to finish the order)

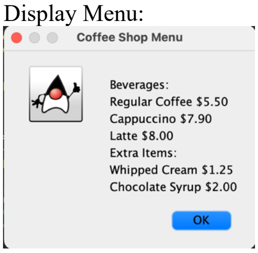



**Output 2:**
Display Order Summary

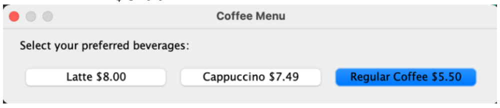


## Testing

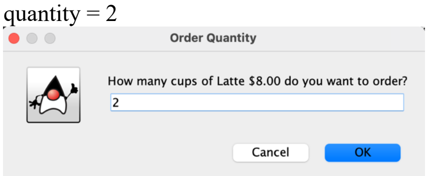
Display Menu:



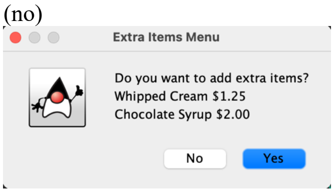

Select latte $8.00



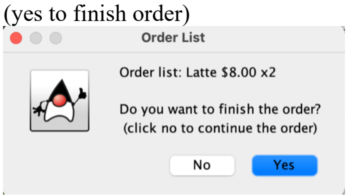

quantity = 2



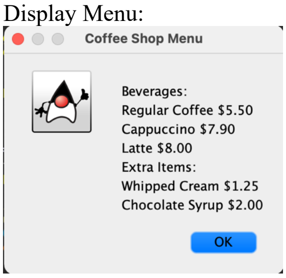

(no)



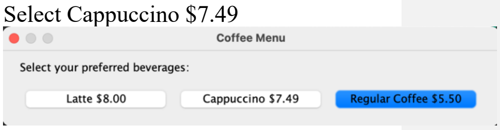

(yes to finish order)



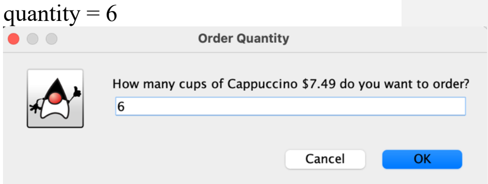

Display Menu:



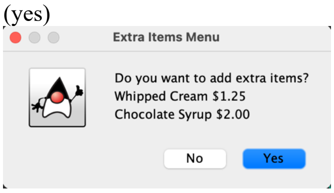

Select Cappuccino $7.49



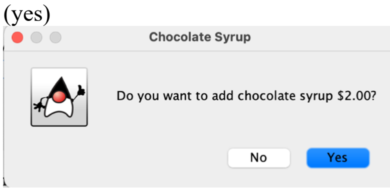

quantity = 6




(yes)

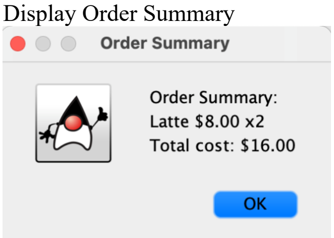



(yes)

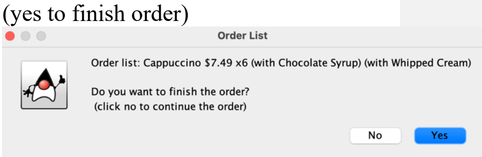



(yes)

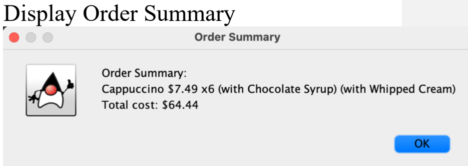
**Output 3:**
Display Order Summary




(yes to finish order)

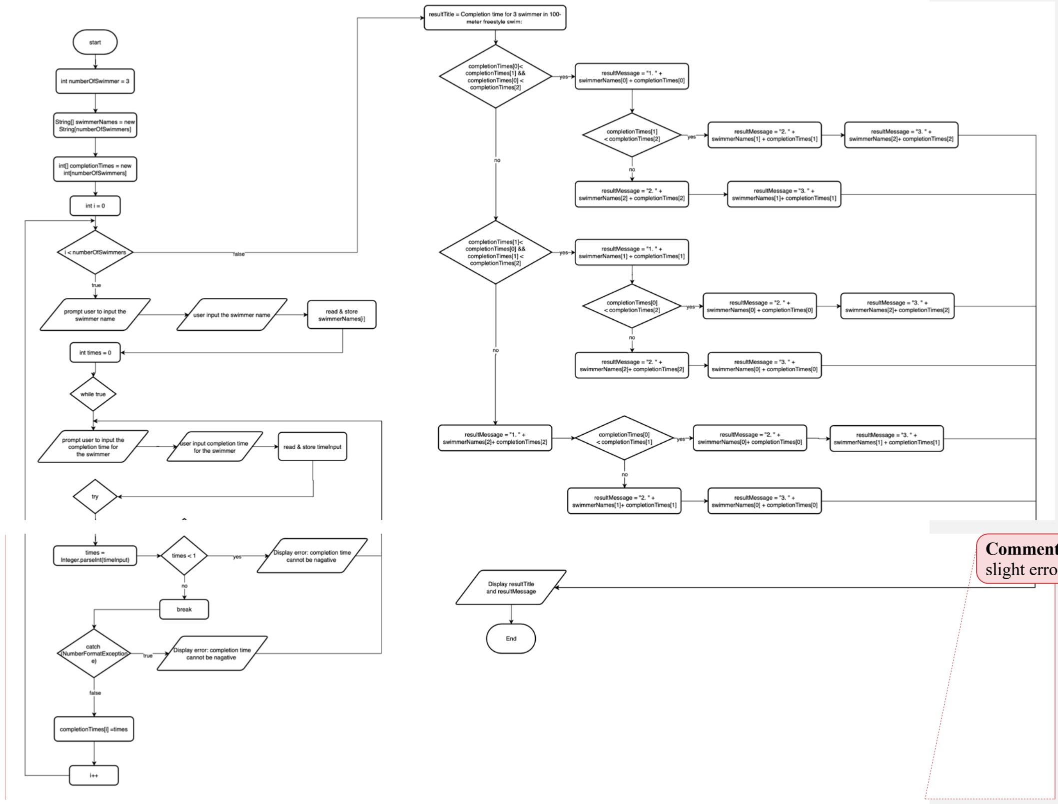



**Output 4:**
Display Order Summary



2. Write a program for a local swimming competition. The program will take input for the names and completion time of five swimmers in a 100-meter freestyle swim in seconds. The program should then display the names of the swimmers in order they finish the race, from the fastest time to the slowest time.

Flowchart https://t.ly/tAWnh (link to view the flowchart)




**Commented [KTST3]:** 1. Same issue with the first part, slight error in the try catch part.

Task 2 source codes

```
/*The purpose of this program is to take input for the names and completion times of 3
swimmer
 *then display the names of the swimmers in order they finish the race, from the fastest time to
the slowest time*/

import javax.swing.JOptionPane;


public class SwimmingCompetition {

	public static void main(String[] args) {

		int numberOfSwimmers = 3;
	    String[] swimmerNames = new String[numberOfSwimmers];
	    int[] completionTimes = new int[numberOfSwimmers];
	    String results, resultTitle;//declare the variable

	    for (int i = 0; i < numberOfSwimmers; i++) {
	swimmerNames[i] = JOptionPane.showInputDialog(null,"Enter the name of swimmer "
+ (i + 1) + ": ","Swimmer Name",1);


	          //ask user to input the completion time
	String timeInput;
	int times = 0;
	while (true) {
	    timeInput = JOptionPane.showInputDialog(null,"Enter the completion time for " +
swimmerNames[i] + " (in seconds): ","Completion Time",1);
	    try {
	        times = Integer.parseInt(timeInput);
	        if (times < 1) {
	            JOptionPane.showMessageDialog(null, "Error: Completion time cannot be
negative. Please enter a valid time.","ERROR",0);
	        }
	        else {
	            break;
	        }
	    } catch  (NumberFormatException e) {
	        JOptionPane.showMessageDialog(null, "Error: Invalid input. Please enter a valid
time.","Error",0);
	    }
	}

	completionTimes[i] =times;

	    }
```

**Commented [KTST4]:** Code is good but did not account for tie between swimmers.

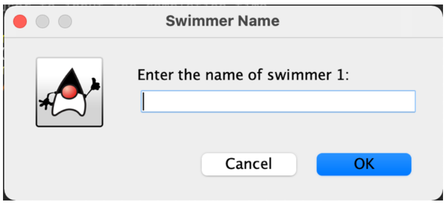
```
        //compare the completion time from the fastest to the slowest
         resultTitle = "Completion time for 3 swimmer in 100-meter freestyle swim: \n";
           if (completionTimes[0] < completionTimes[1] && completionTimes[0] <
completionTimes[2]) {
               results = "1. " + swimmerNames[0] + " (" + completionTimes[0] + " seconds)" +
"\n";
               if (completionTimes[1] < completionTimes[2]) {
                  results += "2. " + swimmerNames[1] + " (" + completionTimes[1] + "
seconds)" + "\n";
                  results += "3. " + swimmerNames[2] + " (" + completionTimes[2] + "
seconds)";
               } else {
                  results += "2. " + swimmerNames[2] + " (" + completionTimes[2] + "
seconds)" +"\n";
                  results += "3. " + swimmerNames[1] + " (" + completionTimes[1] + "
seconds)";
               }
           } else if (completionTimes[1] < completionTimes[0] && completionTimes[1] <
completionTimes[2]) {
               results = "1. " + swimmerNames[1] + " (" + completionTimes[1] + " seconds)" +
"\n";
               if (completionTimes[0] < completionTimes [2]) {
                  results += "2. " + swimmerNames[0] + " (" + completionTimes[0] + "
seconds)" + "\n";
                  results += "3. " + swimmerNames[2] + " (" + completionTimes[2] + "
seconds)";
               }
               else {
                  results += "2. " + swimmerNames[2] + " (" + completionTimes[2] + "
seconds)" + "\n";
                  results += "3. " + swimmerNames[0] + " (" + completionTimes[0] + "
seconds)";
               }
           } else {
               results = "1. " + swimmerNames[2] +" (" + completionTimes[2] + " seconds)" +
"\n";
               if (completionTimes[0] < completionTimes[1]) {
                  results += "2. " + swimmerNames[0] + " (" + completionTimes[0] + "
seconds)" + "\n";
                  results += "3. " + swimmerNames[1] + " (" + completionTimes[1] + "
seconds)";
               }
               else {
                  results += "2. " + swimmerNames[1] + " (" + completionTimes[1] + "
seconds)" + "\n";
                  results += "3. " + swimmerNames[0] + " (" + completionTimes[0] + "
seconds)";
               }
           }
```

```
                JOptionPane.showMessageDialog(null, resultTitle + results, "100-meter Freestyle Swimming Competition Results ", JOptionPane.INFORMATION_MESSAGE);



        }
}
```

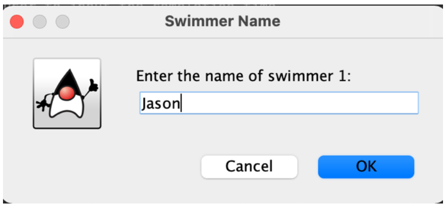
Task 2 Input and Output

**Display Swimmer Name:**

(ask user to enter the name of swimmer 1)

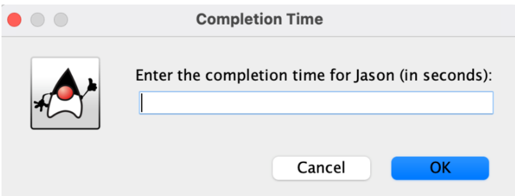


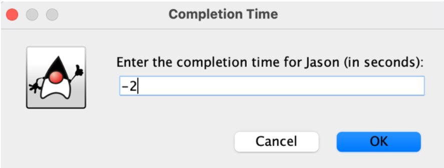
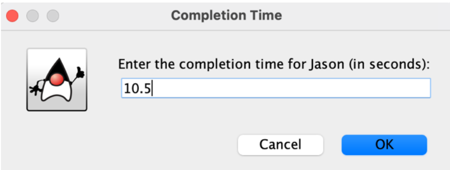
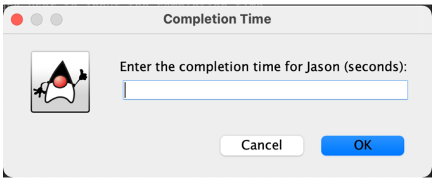
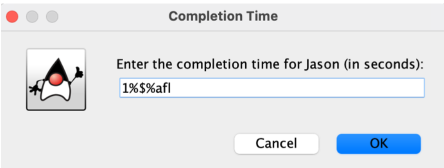

(user enter name of swimmer 1)



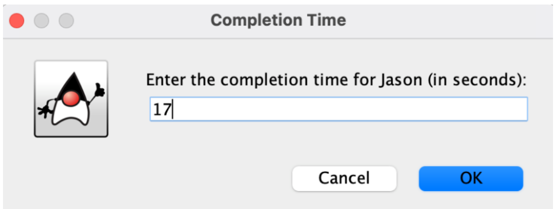

**Display Completion Time:**

(ask user to enter the completion time for swimmer 1)



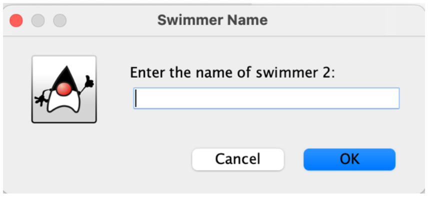
<table>
<tr>
<td>(if user enter a negative value for completion time)<br><br></td>
<td>Display error:<br>(prompt user to enter the time again)<br><br></td>
</tr>
<tr>
<td>(if user enter a decimal value for completion time)<br><br></td>
<td>Display Error:<br>(prompt user to enter the time again)<br><br></td>
</tr>
<tr>
<td>(if input is null)/ (user enter String for completion time)<br><br><br><br></td>
<td>Display Error:<br>(prompt user to enter the time again)<br><br></td>
</tr>
</table>

(if user enter a positive value for completion time)

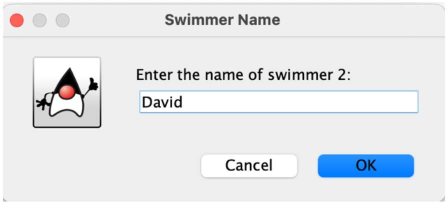


(ask user to enter the name for swimmer 2)

**Display Swimmer Name:**

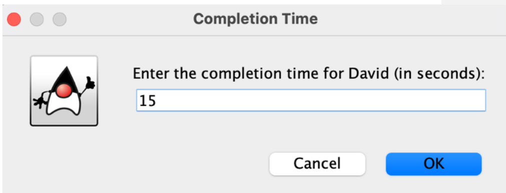



(user enter name for swimmer 2)



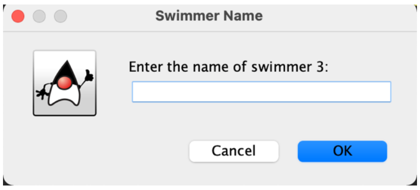

**Display completion time:**

(ask user enter completion time for swimmer 2)

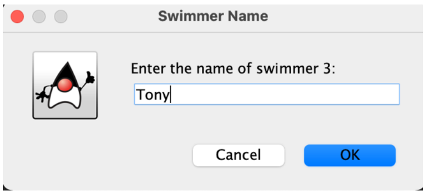



(ask user enter name of swimmer 3)

**Display Swimmer Name:**

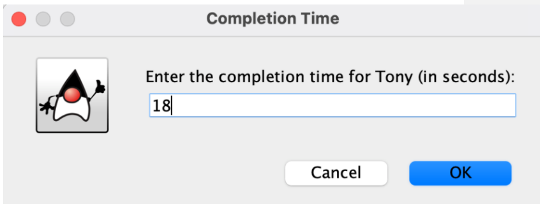



(user enter name for swimmer 3)

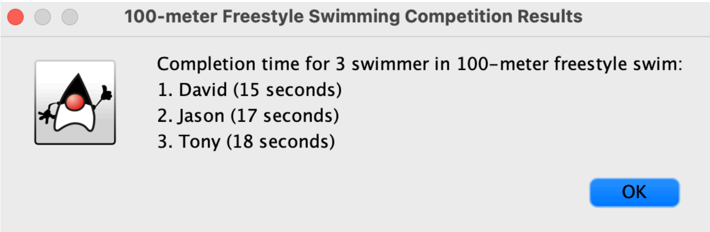



**Display completion time:**

(ask user enter completion time for swimmer 3)



**Display Results:**

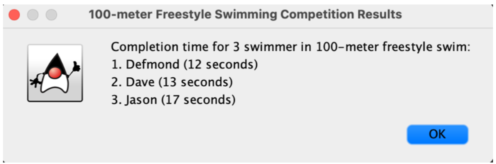



**Testing:**

<table>
<tr>
<td>

**Output 1:**

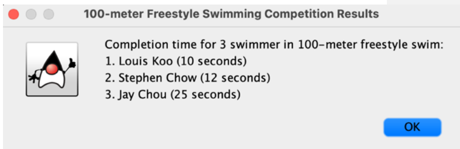
Display Results (from fastest to the slowest)




</td>
<td>

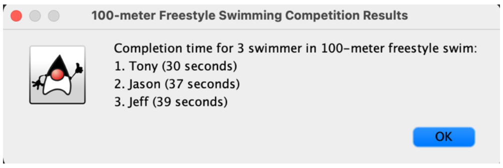
**Output 2:**

Display Results (from fastest to the slowest)



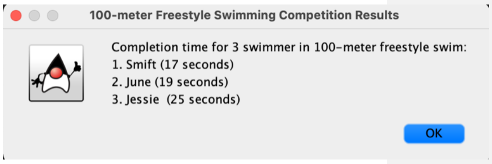

</td>
</tr>
<tr>
<td>

**Output 3:**




</td>
<td>

**Output 4:**




</td>
</tr>
</table>